ALPHA-RAG

Retrieve · Embed · Reason · Respond

NET-02 | ASP.NET Web API with RAG Endpoint

Use Case & System Diagrams Report

Minimal API · ONNX Embeddings · OpenAI / Ollama LLM · Swagger UI

Stack: .NET 8 · ASP.NET Core Minimal API · Microsoft.SemanticKernel · ONNX Runtime · OpenAI / Ollama

Nandhini P · Nikitha S · Kalaiselvi V · Menmathi A

Department of Information Technology

VSB Engineering College

1. System Actors & Use Cases

Alpha-RAG is a six-layer ASP.NET Core service. The actors below interact with the system; the use cases describe what each actor can do through the Minimal API surface.

1.1 Actors

Actor	Description
API Client / Developer	Calls POST /chat through Swagger UI, curl, Postman, or a downstream .NET / JS app.
Knowledge Author	Drops Markdown files into the docs/ folder and triggers ingestion.
Markdown Corpus	The folder of .md documents that grounds every answer.
Embedding Service	ONNX Runtime hosting all-MiniLM-L6-v2 for fully local 384-dim vectors.
Vector Store	In-memory cosine-similarity index with snapshot-to-disk.
OpenAI / Ollama LLM	External or local large-language model that generates the final answer.
Swagger UI	Interactive OpenAPI docs that render request/response examples.

1.2 Use Case Summary

UC#	Use Case	Primary Actor	Brief Description
UC1	Ingest Markdown Docs	Knowledge Author	Scan docs/ and load every .md file with Markdig.
UC2	Chunk & Normalize	Ingestion Service	Sliding-window chunker with configurable size and overlap.
UC3	Generate Embeddings	Embedding Service	ONNX MiniLM produces L2-normalised 384-dim vectors.
UC4	Persist Vector Store	Vector Store	Upsert chunks + vectors and snapshot to a .bin file.
UC5	Receive /chat Request	API Client	Validate ChatRequest payload at the Minimal API endpoint.
UC6	Retrieve Top-K Chunks	RagService	Cosine-similarity search returns the most relevant chunks.
UC7	Build Grounded Prompt	RagService	Render a Scriban template that enumerates chunks as citations.
UC8	Invoke LLM	LLM Layer	Call OpenAI gpt-4o-mini or Ollama llama3 via IChatCompletionService.

UC9	Return Answer + Citations	API Client	Match [n] markers to chunks and serialise ChatResponse.

2. Use Case Diagram

This diagram captures all external actors and the nine use cases within the Alpha-RAG system boundary, including the «includes» and «uses» relationships between them.

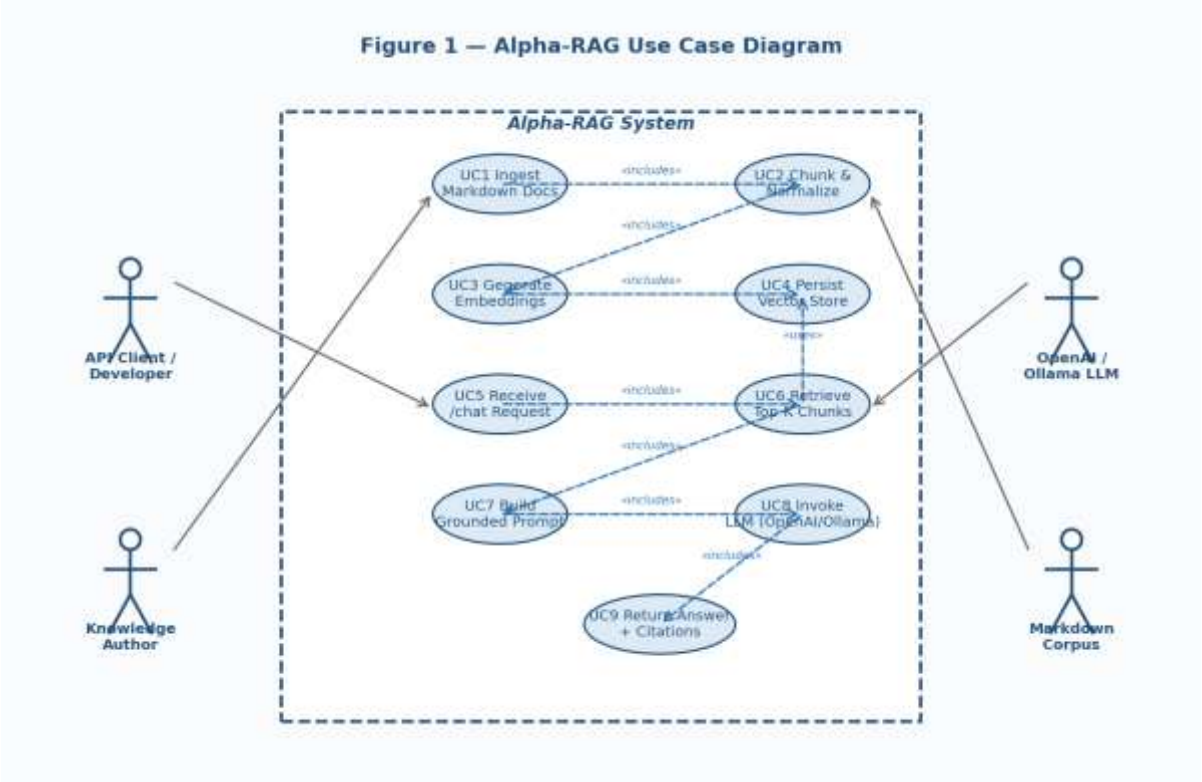

Figure 1 — Use Case Diagram. Nine use cases mapped to four actors with includes / uses relationships.

3. Sequence Diagram

The sequence diagram captures the full message lifecycle for a single /chat call — from JSON request, through embedding and vector retrieval, into the LLM, and back out as a typed ChatResponse with citations.

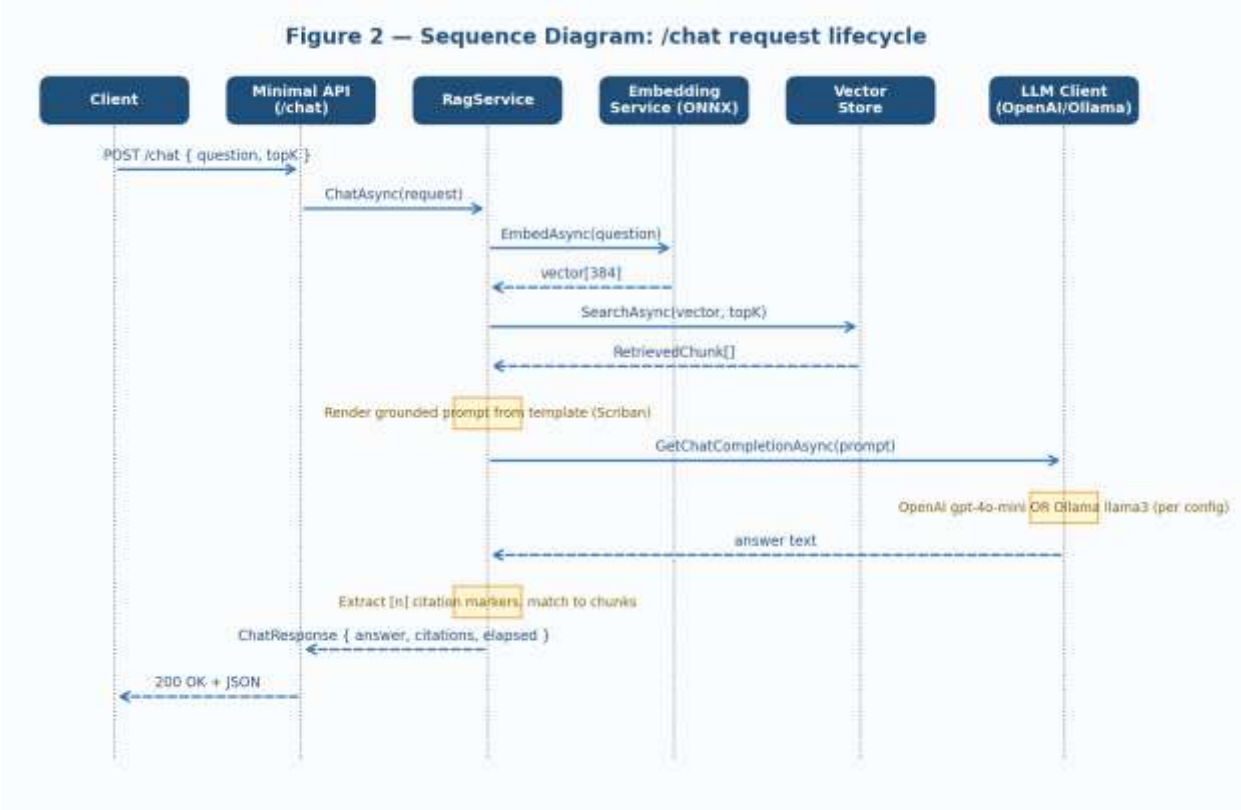

Figure 2 — Sequence Diagram. Lifecycle of one /chat request across six lifelines.

4. Activity / Flowchart Diagram

The activity diagram models Alpha-RAG's decision logic for a single request — input validation, embedding, similarity threshold check, graceful fallback when no relevant context exists, and the LLM-provider branch driven by the UseLocalLlm configuration flag.

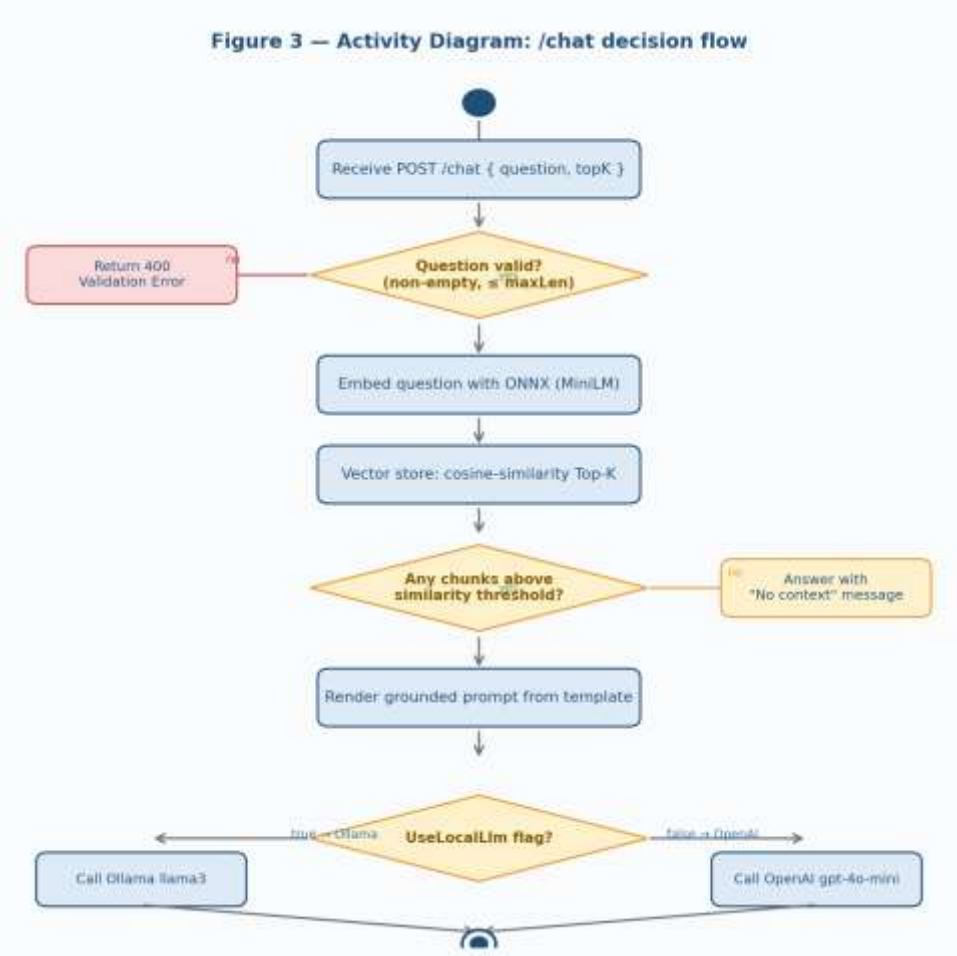

Figure 3 — Activity Diagram. Decision flow for a single /chat invocation.

5. System Architecture Diagram

Alpha-RAG is organised in six layers from the client down to the LLM provider. The embedding and retrieval layers sit entirely in-process, so the service can run fully offline against a local Ollama daemon.

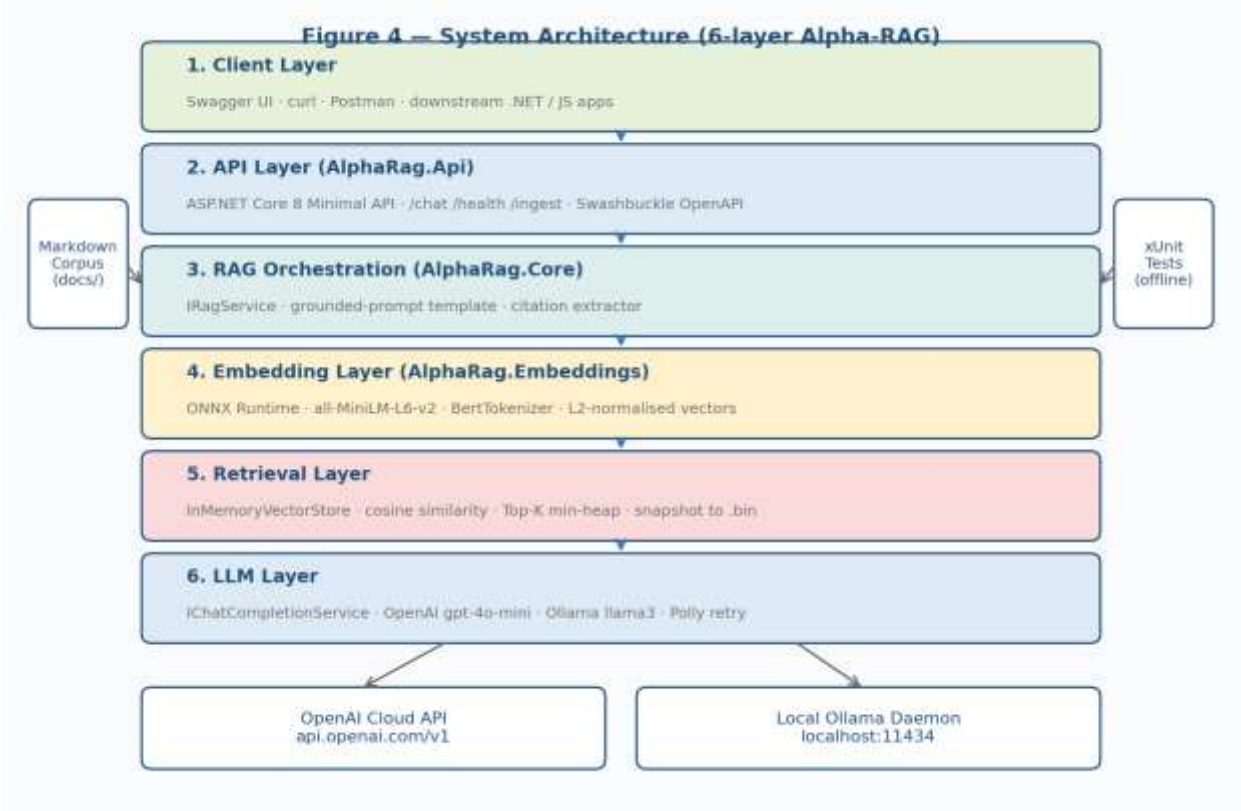

Figure 4 — System Architecture. ASP.NET Core 8 Minimal API · ONNX MiniLM · InMemoryVectorStore · OpenAI / Ollama.

5.1 Layer Summary

#	Layer	Key Components / Technology
1	Client	Swagger UI · curl · Postman · downstream .NET / JS apps.
2	API (AlphaRag.Api)	ASP.NET Core 8 Minimal API · /chat /health /ingest · Swashbuckle.
3	RAG Orchestration (AlphaRag.Core)	IRagService · Scriban grounded-prompt template · citation extractor.
4	Embeddings (AlphaRag.Embeddings)	ONNX Runtime · all-MiniLM-L6-v2 · BertTokenizer · L2 normalisation.
5	Retrieval	InMemoryVectorStore · cosine similarity · Top-K min-heap · binary snapshot.
6	LLM	IChatCompletionService · OpenAI gpt-4o-mini · Ollama llama3 · Polly retry.

6. Ingestion Pipeline

Ingestion is the offline path that turns a folder of Markdown into a searchable vector store. It runs at startup and on demand via POST /ingest, and is incremental — files whose SHA-256 has not changed are skipped.

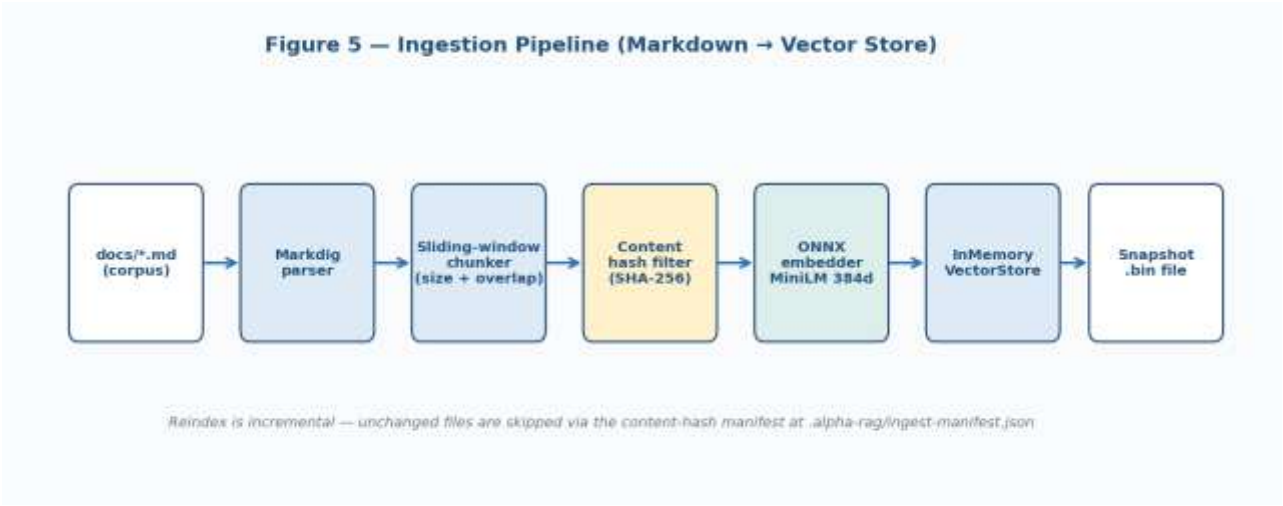

Figure 5 — Ingestion Pipeline. Markdown → chunks → embeddings → vector store → snapshot.

6.1 Stage Details

Stage	Responsibility
Markdig parser	Parse Markdown AST; strip front-matter and preserve headings as chunk metadata.
Sliding-window chunker	Token-aware split using CorpusOptions.ChunkSize and ChunkOverlap.
Content-hash filter	Skip unchanged files via the manifest at .alpha-rag/ingest-manifest.json.
ONNX embedder	Batch inference through MiniLM, mean-pool over the attention mask, L2 normalise.
Vector store upsert	Thread-safe write with ReaderWriterLockSlim to allow concurrent /chat reads.
Binary snapshot	Versioned .bin layout for sub-250 ms warm starts.

7. Component & Interface Diagram

The component diagram shows the four key abstractions of the .NET solution — IRagService, IEmbeddingService, IVectorStore, and IChatCompletionService — together with the immutable ChatRequest / ChatResponse records that flow across the wire.

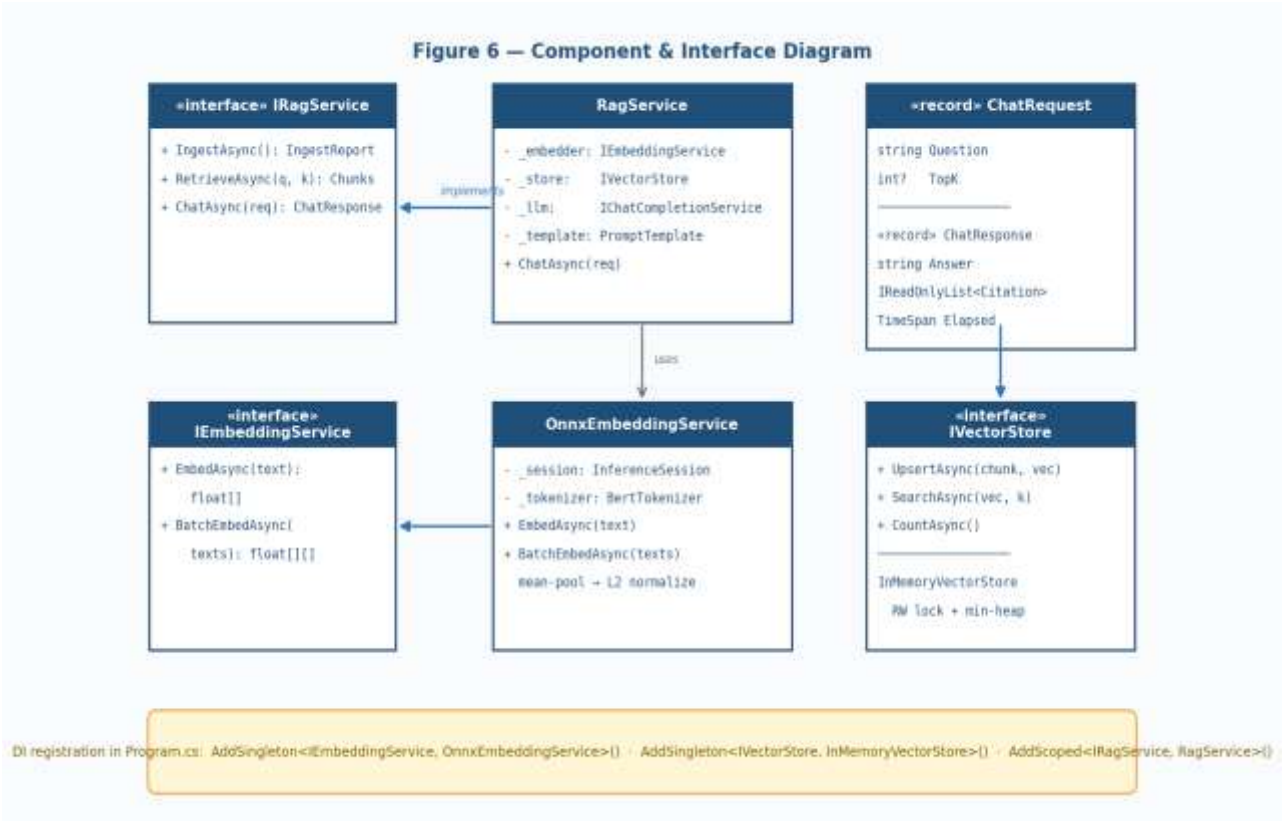

Figure 6 — Component & Interface Diagram. Interfaces, concrete classes, and DI registration.

8. System Inputs

Alpha-RAG accepts heterogeneous inputs through dedicated layers:

• Markdown corpus — .md files in docs/, parsed with Markdig.

• Chat request — JSON ChatRequest { Question, TopK } posted to /chat.

• Ingestion trigger — empty POST /ingest invokes a (re)scan of the corpus.

• Configuration — appsettings.json sections for EmbeddingsOptions, LlmOptions, CorpusOptions, plus User Secrets for the OpenAI API key in Development.

9. Use Case Specifications

9.1 UC5 — Receive /chat Request

Field	Detail
Primary Actor	API Client / Developer
Precondition	Vector store is populated (UC1–UC4 have completed at least once).
Main Flow	POST /chat → validate ChatRequest → invoke IRagService.ChatAsync → serialise ChatResponse → 200.
Exception	Empty Question or Question above maxLen → 400 ValidationProblem.
Postcondition	ChatResponse returned with Answer, Citations, and Elapsed.

9.2 UC6 — Retrieve Top-K Chunks

Field	Detail
Primary Actor	RagService
Input	float[384] question vector + TopK (default 5).
Output	IReadOnlyList<RetrievedChunk> ordered by descending cosine similarity.
Rule	Chunks below the similarity threshold are dropped before prompt assembly.
Postcondition	Retrieved chunks attached to the prompt with [1]..[K] citation markers.

9.3 UC8 — Invoke LLM (OpenAI / Ollama)

Field	Detail
Primary Actor	LLM Layer

Precondition	Grounded prompt rendered; LlmOptions.Provider configured.
Main Flow	IChatCompletionService.GetChatMessageContentAsync with retry + timeout via Polly.
Alt Flow	Provider 5xx → exponential backoff up to 3 attempts → 503 to client if still failing.
Postcondition	Raw answer text returned for citation extraction.

10. Risks & Mitigations

Risk	Mitigation
LLM hallucinates content not in the corpus	Grounded prompt explicitly instructs "answer only from context"; citations are matched back to retrieved chunks before returning.
No relevant chunks above threshold	Service returns a graceful "insufficient context" answer rather than guessing.
OpenAI outage or quota exhaustion	UseLocalLlm flag flips traffic to Ollama llama3 without redeploy.
Stale vector store after corpus edits	Incremental SHA-256 manifest detects changed files and re-embeds only those chunks.
Secret leakage via appsettings	API keys live only in User Secrets (dev) or environment variables (prod); .gitignore excludes the secrets file.
Embedding model drift	Model version pinned in repo; xUnit golden-vector test fails the build if the ONNX output changes.

11. Success Metrics

- p95 /chat latency < 2.0 s end-to-end with OpenAI gpt-4o-mini and a 10 K-chunk corpus.
- ≥ 90% of returned answers carry at least one citation that resolves to a real chunk.
- 0 secrets committed to the repository (enforced by pre-commit scan and CI).
- 100% of public API surface documented in Swagger UI with at least one example payload.
- xUnit suite runs in under 8 seconds with zero network access (UseLocalLlm + StubEmbeddingService).
- Cold-start of the vector store from snapshot under 250 ms for 10 K chunks.